Assign 2

과제 풀이 정리 · Notion 아카이브

Problem 3 : *Sentiment Classification*

3a - Feature(특징)추출

str -> FeatureVector

EX>

"I am what I am" --> {'I': 2, 'am': 2, 'what': 1}

==========

```
def extractWordFeatures(x: str) -> FeatureVector:
x_dict = {}
for y in x.split():
x_dict[y] = x_dict.get(y,0) + 1
return x_dict
```

3b - stochastic gradient descent

|trainExamples|, |validationExamples| : list[tuple]

|featureExtractor| : 함수

|numEpochs|, |eta|

-> WeightVector

call evaluatePredictor() -> error 출력.

output = +1 if the score is precisely 0.

==========

T = TypeVar('T') <- type이 지정되면, 끝까지 그 type을 유지

```
def learnPredictor(trainExamples: List[Tuple[T, int]],
                   validationExamples: List[Tuple[T, int]],
                   featureExtractor: Callable[[T], FeatureVector],
                   numEpochs: int, eta: float) -> WeightVector:

weights = {}   // weight 초기화

for i in range(numEpochs):
```

```
for x,y in trainExamples:
features = featureExtractor(x)

// 마진이 1 미만일 경우 가중치 업데이트
// weigts = weigts + (eta*y)*feature
if dorProduct(weights, features)*y <1
increament(weigts, eta*y, feature)

// 매 에포크 종류 후 훈련/검증 오차 출력
train_error = evaluatePredictor(trainExamples, predictor)
val_error = evaluatePredictor(validationExamples, predictor)
print(f"Epoch {i}: train error = {train_error}, val error = {val_error}")

return weights
```

Problem 4: * Multi-layer Perceptron with ComputationalGraph(그래프)*- nummpy

4a : Node classes for computational graph

- NODE

```
class Node:
    def __init__(self, name: str, *dependencies):
        self.name = name
        self.dependencies = dependencies
                                                    // node that this node depens on (아래)
        self.value = None
        self.grad = None
    def forward(self):
        raise NotImplementedError
    def backward(self):
        raise NotImplementedError
    def grad_initialize(self):
        raise NotImplementedError

================================================

forward -> 노드 자신의 value를 찾음 (값 구하기)

backward -> dependencies의 grad를 구함
```

=dependencies에 대한 grad (파란 글씨 구하기)

grad_initalize -> 기울기 초기화(차원 맞추기)

- computational graph - Matrix multiplication

```
class MatMul(Node):
  """Matrix multiplication"""
def forward(self):
x, y = self.dependencies

self.value = x.value @ y.value

def backward(self):
x, y = self.dependencies

x.grad += self.grad @ y.vlaude.T
y.grad += x.value.T @ self.grad

def grad_initialize(self):
self.grad = np.zeros_like(self.value)
```

- computational graph - Sigmoid

```
class Sigmoid(Node):
"""Sigmoid activaion"""
def forward(self):
x, = self.dependencies    //데이터의 개수가 1개인 것을 강제

self.value = 1/(1 + np.exp(-x.vlaue))

def backward(self):
x, = self.dependencies

x.grad += self.grad * self.value * (1 - self.value)

def grad_initialize(self):
self.grad = np.zeros_like(self.value)
```

4b÷MLPPredictor forward and backward

class MLPPredictor:

def __init__(self, input_size, hidden_size, output_size):
self.W1 = np.random.randn(input_size, hidden_size)
    self.W2 = np.random.randn(hidden_size, output_size)
    self.b1 = np.zeros((1, hidden_size))
    self.b2 = np.zeros((1, output_size))

-> w,b 초기화.
W1 : input_size X hidden_size
W2 : hieend_size X output_size
b1 : 1 X hidden_size
b2 : 1 X output_size

Y = sigmoid(sigmoid(X@W1 + b1)@W2 + b2)
==========
==========
  def forward(self.x):
x_node = Input("x", x.astype(np.float64))
    W1_node = Input("W1", self.W1.astype(np.float64))
    b1_node = Input("b1", self.b1.astype(np.float64))
    W2_node = Input("W2", self.W2.astype(np.float64))
    b2_node = Input("b2", self.b2.astype(np.float64))

  // 각 객체를 생성하는 단계
  z1 = MatMul("z1",x_node, W1_node)
  hidden_input = Add("hidden_input", z1, b1_node)
  hidden = Sigmoid("hidden",hidden_input)
  z2 = MatMul("z2",hidden, W2_node)
  final_input = Add("final_input", z2, b2_node)
  output_node = Sigmoid("output_node", final_input)

  //
  nodes = topological_sort(output_node) //노드 정렬-numpy.ndarray
  for node in nodes:
node.forward()

self.last_nodes = nodes

self.last_output_node = output_node

return output_node.value.squeeze() //-> 예측값.

==========
==========

@staticmethod //-> 클래스 밖에서 정의하는 일반 함수처럼 만듦 "self." 필요 X)
def loss(pred, target):
return (pred-target)**2 //squared loss

==========
==========

pred : predicted probability (0 to 1), (B,)
target : true label, 0 or 1 (B,)
(B,) == 길이가 B인 1차원 배열(Vector)
-> outpur: dictionary of gradient

loss의 gradient를 구하는 것이니,
self.last_output_node.gard = 2(pred-target)이 됨.

---
def backward(self, pred, target):
pred_reshaped = pred.reshape(-1,1)
target_reshaped = target.reshape(-1,1)
  //reshape(n,m) : n=행의 크기, m=열의 크기
    //reshape(행,열)
    //n=-1 -> 전체 데이터 계수를 m으로 나눠서 알아서 크기 만듦.

for node in self.last_nodes:
node.grad_initialize() //초기화

self.last_output_node.grad = 2*(pred_reshaped-target_reshaped)

for node in reversed(self.last_nodes):
node.backward()

for node in self.last_nodes:

if node.name == "W1":

W1_grad = node.grad

if node.name == "b1":

b1_grad = node.grad

if node.name == "W2":

W2_grad = node.grad

if node.name == "b2":

b2_grad = node.grad

gradients = {

"W1" : W1_grad if W1_grad is not None else np.zeros_like(self.W1),

"b1" : b1_grad if b1_grad is not None else np.zeros_like(self.b1),

"W2" : W2_grad if W2_grad is not None else np.zeros_like(self.W2)

"b2" : b2_grad if b2_grad is not None else np.zeros_like(self.b2)

}

return gradient

==========

def update(self, gradients, learning_rate):

self.W1 -= learning_rate * gradients["W1"]

self.b1 -= learning_rate * gradients["b1"]

self.W2 -= learning_rate * gradients["W2"]

self.b2 -= learning_rate * gradients["b2"]

==========

Inputs

X: input features, (N,6), N: number of samples

Y: ture labels, (N,)

epochs: number of epochs to train

learning_rate: step size for weight update

Outputs

loss: squared loss of the last step

---

def train(self, X, Y, epochs=100, learning_rate=0.1):

for i in range(epochs):

for j in range(X.shape[0]): <-행(Row)의 개수

x_sample = X[j:j+1]

y_sample = Y[jj:j+1] -> j번째 행

pred = self.forward(x_sample)

gradients = self.backward(pred, y_sample)

self.update(gradients, learning_rate)

loss = self.loss(pred, y_sample)

==========

def predict(self, x)

return np.round(self.forward(x),4) //->확률 바탕으로, 결괏값 결정

← 인공지능 · 포스텍 목차로